Assignment Number:17.3

K.Akshay

2303A51817

Q.No.	Question	*Expected Time to complete*
1	**Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts** **Lab Objectives:** • Learn how to clean raw datasets using AI-assisted Python scripting. • Apply preprocessing techniques such as handling missing values, encoding categorical data, and normalization. • Automate repetitive data-cleaning tasks with AI-generated code. • Understand how preprocessing impacts model performance. **Task 1 Employee Dataset Cleaning** **Task:** Use AI assistance to clean an employee information dataset. **Instructions:** • Handle missing values in salary, designation, and experience • Remove duplicate employee IDs • Standardize text values (job titles in lowercase) • Encode categorical features (designation, employment_type) **Expected Output:** A clean dataset suitable for HR analytics and prediction tasks. Prompt : Clean the employee dataset for HR analytics by filling missing values (median for salary, "unknown" for designation, mean for experience), removing duplicate employee IDs, standardizing job titles to lowercase, and encoding categorical features (label encoding for designation, one-hot encoding for employment_type). Return the cleaned dataset ready for predictive modeling.	Week9 - Wednesday

Code :

```python
import pandas as pd
import numpy as np

# Create a sample DataFrame (replace this with your actual data loading)
data = {
    'employee_id': [1, 2, 3, 4, 5, 1, 6, 7, 8, 9, 10, 11],
    'salary': [50000, 60000, np.nan, 75000, 55000, 50000, 80000, 62000, np.nan, 70000, 65000, 58000],
    'designation': ['Software Engineer', 'Data Scientist', 'Project Manager', 'Software Engineer', 'HR Specialist', 'Software Engineer', 'Data Scientist', 'Ma
    'experience': [2, 5, 8, 3, np.nan, 2, 6, 4, 10, 7, 5, 3],
    'employment_type': ['Full-time', 'Full-time', 'Contract', 'Full-time', 'Part-time', 'Full-time', 'Full-time', 'Full-time', 'Contract', 'Full-time', 'Part-
}
df = pd.DataFrame(data)

print("Original Dataset Info:")
df.info()
print("\nOriginal Dataset Head:")
display(df.head())
```

```
Original Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   employee_id      12 non-null     int64
 1   salary           10 non-null     float64
```

[2] 0s

```python
# Fill missing 'salary' with the median
median_salary = df['salary'].median()
df['salary'].fillna(median_salary, inplace=True)

# Fill missing 'designation' with 'Unknown'
df['designation'].fillna('Unknown', inplace=True)

# Fill missing 'experience' with the mean
mean_experience = df['experience'].mean()
df['experience'].fillna(mean_experience, inplace=True)

print("Dataset after filling missing values:")
df.info()
display(df.head())
```

```
Dataset after filling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   employee_id      12 non-null     int64
 1   salary           12 non-null     float64
 2   designation      12 non-null     object
 3   experience       12 non-null     float64
 4   employment_type  12 non-null     object
dtypes: float64(2), int64(1), object(2)
```

```
  df['salary'].fillna(median_salary, inplace=True)
/tmp/ipykernel_9839/2275480539.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inp
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea


  df['designation'].fillna('Unknown', inplace=True)
/tmp/ipykernel_9839/2275480539.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an in
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea


  df['experience'].fillna(mean_experience, inplace=True)
```

	employee_id	salary	designation	experience	employment_type
0	1	50000.0	Software Engineer	2.0	Full-time
1	2	60000.0	Data Scientist	5.0	Full-time
2	3	61000.0	Project Manager	8.0	Contract
3	4	75000.0	Software Engineer	3.0	Full-time
4	5	55000.0	HR Specialist	5.0	Part-time

Summary Statistics for the Cleaned Dataset

Gemini

```
print("Summary Statistics of the cleaned DataFrame:")
display(df.describe())
```

Summary Statistics of the cleaned DataFrame:

	employee_id	salary	experience	designation_encoded	employment_type_Contract	employment_type_Full-time	employment_type_Part-time
count	11.000000	11.000000	11.000000	11.000000	11.000000	11.000000	11.000000
mean	6.000000	63363.636364	5.272727	2.090909	0.181818	0.636364	0.181818
std	3.316625	8720.925722	2.370270	1.814086	0.404520	0.504525	0.404520
min	1.000000	50000.000000	2.000000	0.000000	0.000000	0.000000	0.000000
25%	3.500000	59000.000000	3.500000	0.500000	0.000000	0.000000	0.000000
50%	6.000000	61000.000000	5.000000	2.000000	0.000000	1.000000	0.000000
75%	8.500000	67500.000000	6.500000	3.500000	0.000000	1.000000	0.000000
max	11.000000	80000.000000	10.000000	5.000000	1.000000	1.000000	1.000000

Explanantion : employment_type_Contract, employment_type_Full-time, employment_type_Part-time: These are one-hot encoded binary variables (0 or 1). The mean for these columns represents the proportion of employees belonging to that employment type.

Task 2 – Student Performance Data Preprocessing

Task:

Preprocess student performance data using AI assistance.

Instructions:

- Convert exam_date column into datetime format
- Extract new features (month, semester, day)
- Normalize total_marks
- Detect and handle outliers in marks using IQR or Z-score

Expected Output:

A transformed dataset with time-based academic features and normalized marks.

Prompt :

Preprocess the student performance dataset by converting the exam_date column into datetime format, extracting new time-based features (month, semester, day), normalizing total_marks, and detecting/handling outliers in marks using either IQR or Z-score. Return the transformed dataset with academic time features and normalized marks, ready for analysis and prediction tasks.

Code :

```python
import pandas as pd
import numpy as np

# Create a sample DataFrame for student performance
data = {
    'student_id': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115],
    'exam_date': pd.to_datetime(['2023-01-15', '2023-02-20', '2023-03-10', '2023-04-05', '2023-05-12',
                                 '2023-09-01', '2023-10-10', '2023-11-15', '2023-12-01', '2023-01-25',
                                 '2023-03-20', '2023-06-01', '2023-09-20', '2023-02-10', '2023-04-20']),
    'subject': ['Math', 'Science', 'Math', 'English', 'Science', 'Math', 'English', 'Science', 'Math', 'English',
                'Science', 'Math', 'English', 'Science', 'Math'],
    'total_marks': [85, 72, 91, 68, 78, 95, 70, 88, 62, 80, 75, 99, 50, 77, 105], # 105 is an outlier example
    'grade': ['A', 'B', 'A', 'C', 'B', 'A', 'B', 'A', 'D', 'B', 'B', 'A', 'F', 'B', 'A'],
    'attendance_rate': [0.95, 0.88, 0.99, 0.75, 0.92, 0.98, 0.80, 0.96, 0.70, 0.90, 0.85, 1.00, 0.65, 0.89, 0.93]
}
df_students = pd.DataFrame(data)

print("Original Student Dataset Info:")
df_students.info()
print("\nOriginal Student Dataset Head:")
display(df_students.head())
```

```
Original Student Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 6 columns):
 #   Column           Non-Null Count  Dtype
 0   student_id       15 non-null     int64
 1   exam_date        15 non-null     datetime64[ns]
 2   subject          15 non-null     object
 3   total_marks      15 non-null     int64
 4   grade            15 non-null     object
 5   attendance_rate  15 non-null     float64
dtypes: datetime64[ns](1), float64(1), int64(2), object(2)
memory usage: 852.0+ bytes

Original Student Dataset Head:
```

	student_id	exam_date	subject	total_marks	grade	attendance_rate
0	101	2023-01-15	Math	85	A	0.95
1	102	2023-02-20	Science	72	B	0.88
2	103	2023-03-10	Math	91	A	0.99
3	104	2023-04-05	English	68	C	0.75
4	105	2023-05-12	Science	78	B	0.92

+ Code + Text

Step 1: Convert `exam_date` to datetime and extract time-based features

We'll create `month`, `semester`, and `day` columns.

```python
# Ensure 'exam_date' is datetime (if not already)
df_students['exam_date'] = pd.to_datetime(df_students['exam_date'])

# Extract month
df_students['exam_month'] = df_students['exam_date'].dt.month

# Extract day of the week (0=Monday, 6=Sunday)
df_students['exam_day_of_week'] = df_students['exam_date'].dt.dayofweek

# Define semesters (example: Jan-Jun = Semester 1, Jul-Dec = Semester 2)
def get_semester(month):
    if 1 <= month <= 6:
        return 1
    else:
        return 2

df_students['semester'] = df_students['exam_month'].apply(get_semester)

print("Dataset after extracting time-based features:")
df_students.info()
display(df_students.head())
```

```
Dataset after extracting time-based features:
```

```
Dataset after extracting time-based features:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   student_id        15 non-null     int64
 1   exam_date         15 non-null     datetime64[ns]
 2   subject           15 non-null     object
 3   total_marks       15 non-null     int64
 4   grade             15 non-null     object
 5   attendance_rate   15 non-null     float64
 6   exam_month        15 non-null     int32
 7   exam_day_of_week  15 non-null     int32
 8   semester          15 non-null     int64
dtypes: datetime64[ns](1), float64(1), int32(2), int64(3), object(2)
memory usage: 1.1+ KB
```

	student_id	exam_date	subject	total_marks	grade	attendance_rate	exam_month	exam_day_of_week	semester
0	101	2023-01-15	Math	85	A	0.95	1	6	1
1	102	2023-02-20	Science	72	B	0.88	2	0	1
2	103	2023-03-10	Math	91	A	0.99	3	4	1
3	104	2023-04-05	English	68	C	0.75	4	2	1
4	105	2023-05-12	Science	78	B	0.92	5	4	1

```
print("Summary Statistics of the cleaned DataFrame:")
display(df.describe())
```

Summary Statistics of the cleaned DataFrame:

	employee_id	salary	experience	designation_encoded	employment_type_Contract	employment_type_Full-time	employment_type_Part-time
count	11.000000	11.000000	11.000000	11.000000	11.000000	11.000000	11.000000
mean	6.000000	63363.636364	5.272727	2.090909	0.181818	0.636364	0.181818
std	3.316625	8720.925722	2.370270	1.814086	0.404520	0.504525	0.404520
min	1.000000	50000.000000	2.000000	0.000000	0.000000	0.000000	0.000000
25%	3.500000	59000.000000	3.500000	0.500000	0.000000	0.000000	0.000000
50%	6.000000	61000.000000	5.000000	2.000000	0.000000	1.000000	0.000000
75%	8.500000	67500.000000	6.500000	3.500000	0.000000	1.000000	0.000000
max	11.000000	80000.000000	10.000000	5.000000	1.000000	1.000000	1.000000

Explanation; Employee Dataset (df): Missing values were handled, duplicate employee IDs removed, job titles standardized, and categorical features encoded. Summary statistics were also provided.Student Performance Dataset (df_students): exam_date was converted to datetime, time-based features extracted, total_marks normalized, and outliers in marks handled.

Task 3 – Student Health Data Preparation

Task:

Clean and preprocess student health records using AI scripts.

Instructions:

- Handle missing values in height, weight, BMI
- Encode categorical features (medical_condition, fitness_status)
- Scale numerical features using standard scaling
- Split dataset into training and testing sets

Expected Output:

A preprocessed dataset ready for predictive health analysis.

Prompt :

Clean and preprocess the student health dataset by handling missing values in height, weight, and BMI, encoding categorical features such as medical_condition and fitness_status, scaling numerical features using standard scaling, and splitting the dataset into training and testing sets. Return the preprocessed dataset ready for predictive health analysis.

Code :

```python
import pandas as pd
import numpy as np

# Create a sample DataFrame for student health data
data = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    'age': [15, 16, 15, 17, 16, 15, 17, 16, 15, 17, 16, 15, 17, 16, 15],
    'gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male'],
    'height_cm': [165, 170, 160, 175, 168, 162, 178, 172, 158, 173, 166, np.nan, 177, 171, 163],
    'weight_kg': [55, 60, 50, 65, 58, 52, 70, 62, 48, 63, 56, 51, 68, np.nan, 53],
    'bmi': [20.2, 20.8, 19.5, 21.2, 20.5, 19.8, 22.1, 20.9, 19.2, 21.0, 20.3, 19.9, 21.7, 20.6, np.nan],
    'medical_condition': ['None', 'Asthma', 'None', 'Allergies', 'None', 'None', 'Asthma', 'None', 'Allergies', 'None', 'None', 'Diabetes', 'None', 'None', 'As
    'fitness_status': ['Good', 'Excellent', 'Average', 'Good', 'Average', 'Good', 'Excellent', 'Good', 'Poor', 'Excellent', 'Good', 'Average', 'Excellent', 'Go
}
df_health = pd.DataFrame(data)

print("Original Student Health Dataset Info:")
df_health.info()
print("\nOriginal Student Health Dataset Head:")
display(df_health.head())
```

```
Original Student Health Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   student_id         15 non-null     int64
 2   gender             15 non-null     object
 3   height_cm          14 non-null     float64
 4   weight_kg          14 non-null     float64
 5   bmi                14 non-null     float64
 6   medical_condition  15 non-null     object
 7   fitness_status     15 non-null     object
dtypes: float64(3), int64(2), object(3)
memory usage: 1.1+ KB

Original Student Health Dataset Head:
```

	student_id	age	gender	height_cm	weight_kg	bmi	medical_condition	fitness_status
0	1	15	Male	165.0	55.0	20.2	None	Good
1	2	16	Female	170.0	60.0	20.8	Asthma	Excellent
2	3	15	Male	160.0	50.0	19.5	None	Average
3	4	17	Female	175.0	65.0	21.2	Allergies	Good
4	5	16	Male	168.0	58.0	20.5	None	Average

Step 1: Handle missing values

Step 1: Handle missing values

We will fill missing values in `height_cm`, `weight_kg`, and `bmi` with their respective medians.

[12]

```python
# Fill missing 'height_cm', 'weight_kg', 'bmi' with their medians
df_health['height_cm'].fillna(df_health['height_cm'].median(), inplace=True)
df_health['weight_kg'].fillna(df_health['weight_kg'].median(), inplace=True)
df_health['bmi'].fillna(df_health['bmi'].median(), inplace=True)

print("Dataset after filling missing values:")
df_health.info()
display(df_health.head())
```

```
Dataset after filling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   student_id         15 non-null     int64
 1   age                15 non-null     int64
 2   gender             15 non-null     object
 3   height_cm          15 non-null     float64
 4   weight_kg          15 non-null     float64
 5   bmi                15 non-null     float64
 6   medical_condition  15 non-null     object
 7   fitness_status     15 non-null     object
```

```
df_health['height_cm'].fillna(df_health['height_cm'].median(), inplace=True)
/tmp/ipykernel_9839/1014090647.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inp
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea


df_health['weight_kg'].fillna(df_health['weight_kg'].median(), inplace=True)
/tmp/ipykernel_9839/1014090647.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inp
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea


df_health['bmi'].fillna(df_health['bmi'].median(), inplace=True)
```

	student_id	age	gender	height_cm	weight_kg	bmi	medical_condition	fitness_status
0	1	15	Male	165.0	55.0	20.2	None	Good
1	2	16	Female	170.0	60.0	20.8	Asthma	Excellent
2	3	15	Male	160.0	50.0	19.5	None	Average
3	4	17	Female	175.0	65.0	21.2	Allergies	Good
4	5	16	Male	168.0	58.0	20.5	None	Average

```
# If you had a specific target variable 'y', you would define it here:
# y = df_health['target_column']
# X = df_health.drop(columns=['student_id', 'target_column'])

# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)

print(f"Shape of training features (X_train): {X_train.shape}")
print(f"Shape of testing features (X_test): {X_test.shape}")

print("\nFirst 5 rows of X_train:")
display(X_train.head())

Shape of training features (X_train): (12, 14)
Shape of testing features (X_test): (3, 14)

First 5 rows of X_train:
```

	age	height_cm	weight_kg	bmi	gender_Female	gender_Male	medical_condition_Allergies	medical_condition_Asthma	medical_condition_Diabetes	med
13	0.165521	0.430376	-0.134185	0.065900	1.0	0.0	0.0	0.0	0.0	
5	-1.075888	-1.098590	-0.908327	-0.988498	1.0	0.0	0.0	0.0	0.0	
8	-1.075888	-1.778131	-1.527641	-1.779296	0.0	1.0	1.0	0.0	0.0	
2	-1.075888	-1.438361	-1.217984	-1.383897	0.0	1.0	0.0	0.0	0.0	
1	0.165521	0.260491	0.330301	0.329499	1.0	0.0	0.0	1.0	0.0	

Explanation : **Student Performance Dataset (df_students)**: exam_date was converted to datetime, time-based features extracted, total_marks normalized, and outliers in marks handled.

Student Health Dataset (df_health): Missing values were handled, categorical features encoded, numerical features scaled, and the dataset split into training and testing sets.

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Scenario:

A ride-sharing company has trip data with errors.

Requirements:

- Standardize pickup and drop locations
- Fill missing fare amounts with median values
- Remove duplicate ride entries
- Normalize driver ratings
- Generate a summary of data improvements

Expected Output:

A clean dataset ready for operational analysis.

Prompt :

Clean the ride-sharing trip dataset by standardizing pickup and drop locations, filling missing fare amounts with median values, removing duplicate ride entries, normalizing driver ratings, and generating a summary of data improvements. Return the cleaned dataset ready for operational analysis.

Code :

```python
import pandas as pd
import numpy as np

# Create a sample DataFrame for ride-sharing trip data
data = {
    'ride_id': [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1001, 1009, 1010],
    'driver_id': [1, 2, 3, 4, 5, 1, 6, 7, 1, 8, 9],
    'pickup_location': [' Downtown', ' Airport ', 'Midtown', 'downtown', 'Central Park', 'Downtown', 'Airport', 'Midtown', ' Downtown', 'Uptown', 'Financial D
    'dropoff_location': ['Uptown', 'Midtown', 'Downtown', 'Airport', 'Brooklyn', 'Uptown', 'Harlem', 'Queens', 'Uptown', 'Bronx', 'Downtown'],
    'fare_amount': [25.50, 45.00, np.nan, 30.00, 35.00, 25.50, 50.00, np.nan, 25.50, 28.00, 32.00],
    'driver_rating': [4.5, 3.8, 5.0, 4.2, 4.0, 4.5, 4.9, 3.5, 4.5, 4.1, 4.8],
    'trip_distance_miles': [5.2, 15.0, 3.5, 8.1, 7.0, 5.2, 18.0, 6.0, 5.2, 6.5, 4.8],
    'trip_duration_minutes': [20, 35, 15, 25, 22, 20, 40, 18, 20, 21, 16]
}
df_rides = pd.DataFrame(data)

print("Original Ride-Sharing Dataset Info:")
df_rides.info()
print("\nOriginal Ride-Sharing Dataset Head:")
display(df_rides.head())
```

```
Original Ride-Sharing Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   ride_id               11 non-null     int64
 1   driver_id             11 non-null     int64
 2   pickup_location       11 non-null     object
 3   dropoff_location      11 non-null     object
 4   fare_amount           9 non-null      float64
 5   driver_rating         11 non-null     float64
 6   trip_distance_miles   11 non-null     float64
 7   trip_duration_minutes 11 non-null     int64
dtypes: float64(3), int64(3), object(2)
memory usage: 836.0+ bytes

Original Ride-Sharing Dataset Head:
```

	ride_id	driver_id	pickup_location	dropoff_location	fare_amount	driver_rating	trip_distance_miles	trip_duration_minutes
0	1001	1	Downtown	Uptown	25.5	4.5	5.2	20
1	1002	2	Airport	Midtown	45.0	3.8	15.0	35
2	1003	3	Midtown	Downtown	NaN	5.0	3.5	15
3	1004	4	downtown	Airport	30.0	4.2	8.1	25
4	1005	5	Central Park	Brooklyn	35.0	4.0	7.0	22

We will clean these columns by removing leading/trailing whitespace and converting them to title case for consistency.

```python
for col in ['pickup_location', 'dropoff_location']:
    df_rides[col] = df_rides[col].str.strip().str.title()

print("Dataset after standardizing locations:")
display(df_rides.head())
```

```
Dataset after standardizing locations:
```

	ride_id	driver_id	pickup_location	dropoff_location	fare_amount	driver_rating	trip_distance_miles	trip_duration_minutes
0	1001	1	Downtown	Uptown	25.5	4.5	5.2	20
1	1002	2	Airport	Midtown	45.0	3.8	15.0	35
2	1003	3	Midtown	Downtown	NaN	5.0	3.5	15
3	1004	4	Downtown	Airport	30.0	4.2	8.1	25
4	1005	5	Central Park	Brooklyn	35.0	4.0	7.0	22

+ Code + Text

Step 4: Normalize driver ratings

We will use Min-Max scaling to normalize `driver_rating` to a range between 0 and 1.

```python
from sklearn.preprocessing import MinMaxScaler

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Reshape the 'driver_rating' column for scaling
df_rides['driver_rating_normalized'] = scaler.fit_transform(df_rides[['driver_rating']])

print("Dataset after normalizing driver ratings:")
df_rides.info()
display(df_rides.head())
```

```
Dataset after normalizing driver ratings:
<class 'pandas.core.frame.DataFrame'>
Index: 10 entries, 0 to 10
Data columns (total 9 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   ride_id                 10 non-null     int64
 1   driver_id               10 non-null     int64
 2   pickup_location         10 non-null     object
 3   dropoff_location        10 non-null     object
```

```
 6   trip_distance_miles       10 non-null     float64
 7   trip_duration_minutes     10 non-null     int64
 8   driver_rating_normalized  10 non-null     float64
dtypes: float64(4), int64(3), object(2)
memory usage: 800.0+ bytes
```

	ride_id	driver_id	pickup_location	dropoff_location	fare_amount	driver_rating	trip_distance_miles	trip_duration_minutes	driver_rating_normalized
0	1001	1	Downtown	Uptown	25.5	4.5	5.2	20	0.666667
1	1002	2	Airport	Midtown	45.0	3.8	15.0	35	0.200000
2	1003	3	Midtown	Downtown	30.0	5.0	3.5	15	1.000000
3	1004	4	Downtown	Airport	30.0	4.2	8.1	25	0.466667
4	1005	5	Central Park	Brooklyn	35.0	4.0	7.0	22	0.333333

Summary of Data Improvements

Summary of Data Improvements

```
print("Overall Data Improvements:")
print(f"- Locations (pickup_location, dropoff_location) standardized to title case and trimmed whitespace.")
print(f"- Missing fare amounts filled with median value ({median_fare:.2f}).")
print(f"- {initial_rides_count - rides_after_deduplication} duplicate ride entries removed.")
print(f"- Driver ratings normalized to a 0-1 scale, available in 'driver_rating_normalized'.")

print("\nCleaned Dataset Ready for Operational Analysis:")
df_rides.info()
display(df_rides.head())
```

```
Overall Data Improvements:
- Locations (pickup_location, dropoff_location) standardized to title case and trimmed whitespace.
- Missing fare amounts filled with median value (30.00).
- 1 duplicate ride entries removed.
- Driver ratings normalized to a 0-1 scale, available in 'driver_rating_normalized'.

Cleaned Dataset Ready for Operational Analysis:
<class 'pandas.core.frame.DataFrame'>
Index: 10 entries, 0 to 10
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   ride_id                   10 non-null     int64
 1   driver_id                 10 non-null     int64
 2   pickup_location           10 non-null     object
```


```
 2   pickup_location           10 non-null     object
 3   dropoff_location          10 non-null     object
 4   fare_amount               10 non-null     float64
 5   driver_rating             10 non-null     float64
 6   trip_distance_miles       10 non-null     float64
 7   trip_duration_minutes     10 non-null     int64
 8   driver_rating_normalized  10 non-null     float64
dtypes: float64(4), int64(3), object(2)
memory usage: 800.0+ bytes
```

	ride_id	driver_id	pickup_location	dropoff_location	fare_amount	driver_rating	trip_distance_miles	trip_duration_minutes	driver_rating_normalized
0	1001	1	Downtown	Uptown	25.5	4.5	5.2	20	0.666667
1	1002	2	Airport	Midtown	45.0	3.8	15.0	35	0.200000
2	1003	3	Midtown	Downtown	30.0	5.0	3.5	15	1.000000
3	1004	4	Downtown	Airport	30.0	4.2	8.1	25	0.466667
4	1005	5	Central Park	Brooklyn	35.0	4.0	7.0	22	0.333333

- Explanation : **ide_id**: Unique identifiers, ranging from 1001 to 1010.
- **driver_id**: IDs for drivers, ranging from 1 to 9.
- **fare_amount**: The average fare is 33.10, with a standard deviation of 8.18. Fares range from 25.50 to 50.00.
- **driver_rating**: The average driver rating is 4.33, with ratings spanning from 3.5 to 5.0.
- **trip_distance_miles**: The mean trip distance is 7.93 miles, with distances from 3.5 to 18.0 miles.
- **trip_duration_minutes**: The average trip duration is 23.2 minutes, with durations ranging from 15 to 40 minutes.
- **driver_rating_normalized**: This column shows the driver ratings scaled to a 0-1 range, with an average of 0.55.